

Adaptive Control of Asymmetric Payloads in Rigid Multi-Quadrotor Systems Using PINNs

Miqdad Abdurrahman, Anggera Bayuwindra, Hilton Tnunay

Sekolah Teknik Elektro dan Informatika (STEI)

Institut Teknologi Bandung (ITB)

Bandung, Indonesia

Email: 23224011@mahasiswa.itb.ac.id, bayuwindra@itb.ac.id, hilton.tnunay@itb.ac.id

Abstract—Cooperative multi-quadrotor systems rigidly attached to a common frame excel at heavy-lift transportation. However, uncertain payload Centers of Gravity (CoG) introduce continuous disturbance torques, degrading geometric controller performance and causing severe lateral drift. This paper introduces an in-the-loop Physics-Informed Neural Network (PINN) parameter estimation framework to dynamically identify unknown CoG shifts in-flight. Unlike classical batch least-squares or Concurrent Learning frameworks that require persistent excitation or mathematically full-rank history stacks, the proposed PINN utilizes continuous gradient descent over a short-term receding horizon. To avoid artificial coordinate singularities, the network formulates translational and rotational physics residuals using Euler-Lagrange and Euler-Poincaré equations directly on the Special Orthogonal group, $SO(3)$. The estimated parameters continuously update the control allocation matrix, enabling adaptive thrust reallocation. Simulations at 240 Hz confirm that the PINN estimator rapidly converges, arrests transient lateral drift, and successfully restores trajectory tracking stability.¹

Index Terms—Multi-Agent Systems, Physics-Informed Neural Networks, Cooperative Payload Transport, Parameter Estimation, Unmanned Aerial Vehicles.

I. INTRODUCTION

A. Motivation and Related Works

Cooperative multi-quadrotor systems provide scalable, high-capacity solutions for payload transportation [1], [2]. While cable-suspended payloads [3], [4] are common, rigidly attaching quadrotors to a common frame allows the distributed system to be controlled as a single composite rigid body [2]. However, this geometric simplification assumes a perfectly symmetrical mass distribution [5], [6]. In practice, uncertain payload Centers of Gravity (CoG) introduce unmodeled disturbance torques, causing severe lateral drift that may lead to crash [7], [8].

Recent mitigations include structural hardware optimization—such as physically canting rotors to artificially increase yaw authority [2]—and algorithmic adaptation. While digital twin (DT) model predictive control [8] and Concurrent Learning [7] effectively adapt to unknown parameters, they often suffer from computational latency or require persistent excitation during vulnerable transient phases.

To mitigate these unmodeled disturbances, recent literature has explored online parameter estimation and advanced control

architectures. Classical approaches, such as the batch and recursive least-squares algorithms utilized in [5], require strict persistent excitation to prevent regressor matrix singularities. To overcome the need for continuous excitation, Concurrent Learning (CL) frameworks [7] have been introduced. While CL successfully adapts to unknown parameters, convergence is mathematically guaranteed only when the recorded history stack achieves full column rank. This strict linear independence requirement leaves the multi-agent system (MAS) highly vulnerable to lateral drift during the initial transient flight phase. Alternatively, computationally intensive frameworks leveraging Model Predictive Control (MPC) synchronized with Digital Twins (DT) [8] have been proposed to estimate the state of irregular payloads. However, this architecture inherently introduces network and computational latency between the physical controller and the DT simulator, degrading the real-time responsiveness required for aggressive stabilization.

B. Contributions

To bridge the gap between rapid parameter estimation and adaptive control, we introduce a novel framework for rigid multi-quadrotor systems. The primary contributions of this work are:

- 1) **Rank-Free PINN Estimation:** We develop an in-the-loop PINN that dynamically identifies the unknown CoG shift of an asymmetric payload utilizing a short-term receding horizon. By leveraging continuous gradient descent rather than block matrix inversion, the estimator bypasses the strict full-rank data requirements of classical Concurrent Learning, enabling rapid parameter convergence without relying on highly-variant persistent excitation.
- 2) **Singularity-Free Manifold Optimization:** The network's loss function embeds translational Euler-Lagrange and rotational Euler-Poincaré equations directly on the $SO(3)$ manifold. This energy-based formulation ensures globally valid 6-DOF physics constraints while entirely eliminating the artificial singularities (gimbal lock) inherent to minimal coordinate representations.
- 3) **Dynamic Control Integration:** The continuously updated CoG parameters are fed directly into the geometric

¹The source code for the PINN estimator and simulation environment is available at: https://github.com/miqdude/pinn_invese_estimator_conf_repo

control allocation matrix. This closed-loop integration empowers the composite rigid body to adaptively reallocate thrust and reject disturbance torques during flight, restoring tracking stability without requiring physically canted hardware.

II. PRELIMINARIES

A. Cooperative Payload Transportation Dynamics

In cooperative aerial transportation, multiple unmanned aerial vehicles (UAVs) are physically coupled to a shared payload to increase total lift capacity and system redundancy [1]. When rigidly attached, the distributed Multi-Agent System (MAS) kinematically functions as a single composite rigid body [2]. While this mechanical coupling fundamentally eliminates the need for complex inter-agent collision avoidance algorithms, it introduces significant dynamic challenges [7], [8].

The stability and navigation of the MAS become a centralized rigid-body control problem. Let the inertial world frame be denoted by $\mathcal{W}$ and the composite body-fixed frame by $\mathcal{B}$. The state of this composite system is defined by its global position $\mathbf{r} \in \mathbb{R}^3$ and its spatial orientation $\mathbf{R} \in SO(3)$. Control authority is achieved via an allocation matrix A , which maps the individual scalar thrusts generated by each quadrotor into a singular composite wrench acting on $\mathcal{B}$. Consequently, safe trajectory tracking depends entirely on accurate geometric mapping, where the primary dynamic unknowns dictating system stability are the payload's shifting Center of Gravity (CoG) and the resulting inertia tensor.

B. Physics-Informed Parameter Estimation

Physics-Informed Neural Networks (PINNs) represent a paradigm shift in scientific machine learning by seamlessly embedding known physical laws into the deep learning optimization process. Unlike purely data-driven models that rely exclusively on large empirical datasets to learn system behaviors implicitly, PINNs utilize governing Ordinary Differential Equations (ODEs) as explicit regularizers within their loss function [9], [10].

In the context of aerial robotics or control system in generic, PINNs are highly effective for solving *inverse problems* [11]. By using continuous time t as the primary input, the network predicts the kinematic states of the MAS. These predictions are then evaluated against the strict mathematical boundaries of Newton-Euler rigid body dynamics. By minimizing the residual discrepancy between the network's predictions, the measured flight data, and these physical constraints, the PINN can dynamically infer unmodeled system parameters—such as the unknown CoG offset—directly during flight, providing a critical bridge between physical mechanics and adaptive control [12].

III. SYSTEM DESCRIPTION

A. System Modeling

We define the world frame as $\mathcal{W}$ and the body frame as $\mathcal{B}$, position is $\mathbf{r} = [x, y, z]^T$, and orientation is defined by Euler angles $\eta = (\phi, \theta, \psi)$:

$$\mathcal{W} = [x_{\mathcal{W}}, y_{\mathcal{W}}, z_{\mathcal{W}}] \quad (1)$$

$$\mathcal{B} = [x_{\mathcal{B}}, y_{\mathcal{B}}, z_{\mathcal{B}}] \quad (2)$$

The MAS we implement is a centralized control derived from [5] commanding the drones by giving the control input $\boldsymbol{\tau} = [F_B, M_{xB}, M_{yB}, M_{zB}]^T$ relates to individual thrust inputs $\mathbf{u} = [F_1, F_2, F_3, F_4]^T$ via the allocation matrix $A \in \mathbb{R}^{4 \times 4}$:

$$\boldsymbol{\tau} = [F_B, M_{xB}, M_{yB}, M_{zB}]^T = A\mathbf{u} \quad (3)$$

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ y_1 & y_2 & y_3 & y_4 \\ -x_1 & -x_2 & -x_3 & -x_4 \\ c_1 & c_2 & c_3 & c_4 \end{bmatrix} \quad (4)$$

The rows of A denote total thrust, roll moment, pitch moment, and yaw reactive torque, respectively. Required individual thrusts are derived by inversion:

$$\mathbf{u} = A^{-1} [F_B \ M_{xB} \ M_{yB} \ M_{zB}] \quad (5)$$

B. Centralized Control Design

Based on [5], the controller separates position and attitude tracking. Position and velocity errors define commanded acceleration $\ddot{\mathbf{r}}_{cmd}$:

$$\mathbf{e}_p = \mathbf{r}_{des} - \mathbf{r}, \quad \mathbf{e}_v = \dot{\mathbf{r}}_{des} - \dot{\mathbf{r}} \quad (6)$$

$$\ddot{\mathbf{r}}_{cmd} = K_p \mathbf{e}_p + K_d \mathbf{e}_v + \ddot{\mathbf{r}}_{ff} \quad (7)$$

Desired roll (ϕ^{des}), pitch (θ^{des}), and total thrust (F_B^{des}) are extracted via small-angle approximation:

$$\begin{aligned} \phi^{des} &= \frac{1}{g}(\ddot{r}_x^{des} \sin \psi_T - \ddot{r}_y^{des} \cos \psi_T) \\ \theta^{des} &= \frac{1}{g}(\ddot{r}_y^{des} \cos \psi_T + \ddot{r}_x^{des} \sin \psi_T) \\ F_B^{des} &= m(\ddot{r}_z^{des} + g) \end{aligned} \quad (8)$$

Inner-loop desired moments $\mathbf{M}_B^{des}$ regulate orientation against angular velocities (p, q, r):

$$\begin{aligned} M_{xB}^{des} &= k_{p,\phi}(\phi^{des} - \phi) + k_{d,\phi}(p^{des} - p) \\ M_{yB}^{des} &= k_{p,\theta}(\theta^{des} - \theta) + k_{d,\theta}(q^{des} - q) \\ M_{zB}^{des} &= k_{p,\psi}(\psi^{des} - \psi) + k_{d,\psi}(r^{des} - r) \end{aligned} \quad (9)$$

C. Dynamic Thrust Allocation

An asymmetric payload shifts the CoG, altering the effective moment arms. Let $\mathbf{d}_{\text{cog}} = [\hat{x}_{\text{cog}}, \hat{y}_{\text{cog}}]^T$ denote the estimated offset. Redefining the lever arms as $\Delta x_i = x_i - \hat{x}_{\text{cog}}$ and $\Delta y_i = y_i - \hat{y}_{\text{cog}}$, the allocation matrix adapts dynamically:

$$A_{\text{final}} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ \Delta y_1 & \Delta y_2 & \Delta y_3 & \Delta y_4 \\ -\Delta x_1 & -\Delta x_2 & -\Delta x_3 & -\Delta x_4 \\ c_1 & c_2 & c_3 & c_4 \end{bmatrix} \quad (10)$$

Control inputs are computed via the Moore-Penrose pseudo-inverse to handle transient rank deficiencies:

$$\mathbf{u} = A_{\text{final}}^+ \boldsymbol{\tau} \quad (11)$$

IV. ESTIMATOR DESIGN

A. PINN Architecture

To address 6-DOF dynamics, our PINN framework uses time t to solve time-dependent ODEs [10], [11], [13]. The network structure consists of fully-connected hidden layers with size of 64 per layer. Then, it connects to the output layer that will estimate the 12 states of the body which will be described in the next section. The structure of the network is illustrated in 1. The states will be used to minimize physics-induced residuals which allows the network to identify payload parameters and feed them back into the allocation matrix.

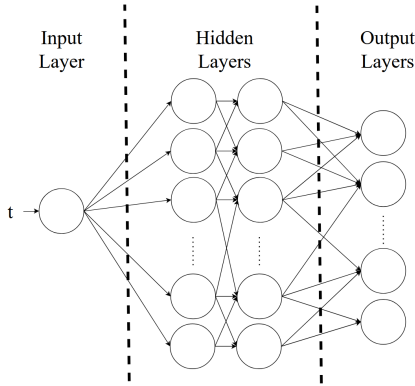


Fig. 1. Illustration of the PINNs Architecture

B. Loss Function Formulation

The neural network acts as a surrogate model mapping continuous time t to the predicted 6-DOF kinematic state of the system. We define the complete state vector $\mathbf{S}$, comprising the position, linear velocity, attitude, and angular velocity:

$$\mathbf{S} = [\mathbf{r}^T, \dot{\mathbf{r}}^T, \boldsymbol{\eta}^T, \boldsymbol{\omega}^T]^T \quad (12)$$

To train the network, we formulate a composite loss function $\mathcal{L}_{\text{total}}$ [12], [13] that simultaneously penalizes empirical data deviations and violations of rigid-body dynamics. This objective function is defined as a weighted sum of the data-driven

observation loss ($\mathcal{L}_{\text{data}}$), the translational physics residual ($\mathcal{L}_{\text{trans}}$), and the rotational physics residual ($\mathcal{L}_{\text{rot}}$):

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{data}} + \lambda_2 \mathcal{L}_{\text{trans}} + \lambda_3 \mathcal{L}_{\text{rot}} \quad (13)$$

where the hyperparameters λ_1, λ_2 , and λ_3 balance the gradient contributions of each loss term.

The data loss ensures the network's predictions anchor to the physical reality measured by the onboard sensors. It is computed as the Mean Squared Error (MSE) over a temporal window of N collocation points:

$$\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_{i=1}^N \left\| \hat{\mathbf{S}}_i - \mathbf{S}_{\text{true},i} \right\|^2 \quad (14)$$

To enforce the translational physical constraints, we formulate a residual loss based on the Euler-Lagrange equations [14]. Let $L = T_{\text{trans}} - V$ denote the system Lagrangian, where T_{trans} is the kinetic energy and V is the potential energy. The translational dynamics must satisfy:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \mathbf{v}} \right) - \frac{\partial L}{\partial \mathbf{r}} = \mathbf{F}_{\text{ext}} \quad (15)$$

Evaluating the kinetic inertial term yields $m\dot{\mathbf{v}}$. Rather than analytically deriving the potential gradient, we leverage PyTorch's Automatic Differentiation (Autograd) engine to dynamically compute $\frac{\partial V}{\partial \mathbf{r}}$ during the forward pass. The resulting translational physics loss minimizes the residual of these forces across the temporal window:

$$\mathcal{L}_{\text{trans}} = \frac{1}{N} \sum_{i=1}^N \left\| m\dot{\mathbf{v}}_i + \frac{\partial V}{\partial \mathbf{r}_i} - \mathbf{F}_{\text{ext},i} \right\|^2 \quad (16)$$

While translational dynamics are straightforward, evaluating rotational dynamics using minimal coordinates (e.g., Euler angles) introduces mathematical singularities, commonly known as gimbal lock, at extreme pitch or roll angles. To ensure global mathematical validity, we evaluate the rotational dynamics directly on the Special Orthogonal group, $SO(3)$. The rotational kinetic energy of the rigid multi-agent system is defined as:

$$T_{\text{rot}} = \frac{1}{2} \boldsymbol{\omega}^T \mathbf{J} \boldsymbol{\omega} \quad (17)$$

where $\mathbf{J} \in \mathbb{R}^{3 \times 3}$ is the estimated inertia tensor. By applying the Lagrange-d'Alembert principle to the $SO(3)$ manifold, we obtain the coordinate-free Euler-Poincaré equations:

$$\frac{d}{dt} \left(\frac{\partial T_{\text{rot}}}{\partial \boldsymbol{\omega}} \right) + \boldsymbol{\omega} \times \frac{\partial T_{\text{rot}}}{\partial \boldsymbol{\omega}} = \boldsymbol{\tau}_{\text{ext}} \quad (18)$$

Recognizing that the partial derivative $\frac{\partial T_{\text{rot}}}{\partial \boldsymbol{\omega}} = \mathbf{J} \boldsymbol{\omega}$ represents the system's angular momentum, equation (18) resolves into the Newton-Euler formulation for rigid body rotation. In our cooperative framework, the total external torque $\boldsymbol{\tau}_{\text{ext}}$ comprises the known applied control wrench $\boldsymbol{\tau}_{\text{app}}$ and the unknown disturbance torque $\boldsymbol{\tau}_{\text{dist}}$ generated by the payload's

CoG offset. The rotational physics residual evaluated during the forward pass is thus formulated as:

$$\text{Res}_{rot} = (\hat{\mathbf{J}}\dot{\hat{\boldsymbol{\omega}}} + \hat{\boldsymbol{\omega}} \times (\hat{\mathbf{J}}\hat{\boldsymbol{\omega}})) - (\boldsymbol{\tau}_{app} - \boldsymbol{\tau}_{dist}) \quad (19)$$

Finally, the rotational loss penalizes the mean squared error of this residual, forcing the neural network to implicitly map the physical discrepancies to the true disturbance parameters:

$$\mathcal{L}_{rot} = \frac{1}{N} \sum_{i=1}^N \|\text{Res}_{rot,i}\|^2 \quad (20)$$

V. SIMULATION AND RESULTS

A. Simulation Environment and Setup

Simulations were executed in PyBullet at $\Delta t = 1/240$ s (240 Hz). Quadrotor thrust is capped at 50 N, and gravity is $g = 9.81$ m/s². The unladen frame mass is $m_f = 6.8$ kg, carrying a $m_p = 1.0$ kg payload ($m_{tot} = 7.8$ kg). Each 60-second episode initializes with a uniformly sampled CoG offset:

$$d_{x,cog}, d_{y,cog} \sim \mathcal{U}(-0.15, 0.15) \text{ meters} \quad (21)$$

Flights exceeding a 2.0 m Euclidean distance error are terminated.

B. Simulation Loop

The Software-in-the-Loop (SITL) architecture interleaves a 240 Hz geometric attitude controller with a 30 Hz PINN estimator, detailed in Algorithm 1.

C. Optimization Parameters

The PINN utilizes a receding horizon buffer of $W = 50$ transitions, executing 3 gradient optimization steps every 5 simulation iterations (Table I).

TABLE I
SYSTEM AND SIMULATION PARAMETERS

Parameter	Symbol	Value
Simulation/Control Rate	f	240 Hz
Simulation Time-Step	Δt	1/240 s
Simulation Total Episodes	E_{max}	10
Simulation Duration	t_{sim}	90 s
Base Frame Mass	m_f	6.8 kg
Payload Mass	m_p	1.0 kg
Total System Mass	m_{tot}	7.8 kg
Max Thrust per Drone	T_{max}	50 N
Max Distance Error	e_{max}	2.0 m
Unknown Payload Offset	d_{cog}	$[-0.15, 0.15]$ m
PINN Window Size	W	50 steps
Optimization Frequency	N_{opt}	5 steps
Gradient Steps per Cycle	S_{grad}	3 steps

Algorithm 1 Simulation Loop with Control and Online Parameters Estimation

Require: Episode maximum E_{max}
Require: Simulation duration T_{max} , Physics time step Δt
Require: PINN update interval $N = 8$ (approx. 30 Hz)
Require: PINN optimization steps S_{steps}
Require: Replay buffer $\mathcal{B}$ with capacity $W = 50$
Require: Initial parameter guesses: $\hat{d}_x, \hat{d}_y, \hat{\mathbf{J}}, \hat{m}$

- 1: Init current episode e_{now}
- 2: **while** $e_{now} \leq E_{max}$ **do**
- 3: Initi quadrotor states $\mathbf{x}_0$
- 4: Initi PINN $\mathcal{N}_\theta$ with random weights θ
- 5: Simulation time $t \leftarrow 0$
- 6: Step counter $k \leftarrow 0$
- 7: **while** $t \leq T_{max}$ **do**
- 8: Compute target states $\mathbf{p}_d, \mathbf{v}_d, \mathbf{a}_d, \psi_d, \dot{\psi}_d$ at t
- 9: Current state $\mathbf{x}_k = (\mathbf{p}, \mathbf{v}, \mathbf{R}, \boldsymbol{\omega})$
- 10: Append $\mathbf{x}_k$ and rotor thrusts $\mathbf{T}_k$ to buffer $\mathcal{B}$
- 11: **if** $k \pmod N == 0$ **and** $|\mathcal{B}| == W$ **then**
- 12: Compute composite loss $\mathcal{L}_{total}$ over $\mathcal{B}$
- 13: Optimize θ for S_{steps}
- 14: Extract parameters $\hat{d}_x, \hat{d}_y, \hat{\mathbf{J}}, \hat{m}$ from $\mathcal{N}_\theta$
- 15: **end if**
- 16: Calculate desired $\mathbf{F}_{des}, \mathbf{M}_{des}$
- 17: Construct $\mathbf{A}_{final}$ for offsets $\hat{d}_x, \hat{d}_y$
- 18: Quadrotors commands $\mathbf{u} \leftarrow \mathbf{A}_{final}^{-1} [\mathbf{F}_{des}^T, \mathbf{M}_{des}^T]^T$
- 19: Apply saturated commands $\mathbf{u}$ to physics engine
- 20: $t \leftarrow t + \Delta t$
- 21: $k \leftarrow k + 1$
- 22: **end while**
- 23: **end while**

VI. RESULTS AND DISCUSSION

A. Parameter Convergence and Estimation

Across 10 distinct simulation episodes, each initialized with newly randomized payload offset parameters, the proposed framework maintained continuous stability without a single failure or crash. Notably, optimal tracking performance was achieved during the very first episode, demonstrating that the system possesses robust, immediate adaptation capabilities and does not rely on multi-episode historical training to function. The PINN effectively minimized the physics residuals formulated in Eqs. (13, 16, 20) as shown in Fig. 3. While transient spikes in the loss function occurred during aggressive maneuvers, these residuals rapidly decreased. This confirms that the coupled PINN and controller can dynamically compensate for varying payload offsets regardless of instantaneous trajectory demands.

Operating over the receding temporal window, the network rapidly mapped the unmodeled disturbance torques to spatial mass asymmetry without requiring persistent excitation maneuvers as illustrated in Fig. 4, the composite vehicle experienced an immediate lateral drift along the positive X-axis upon takeoff due to the payload asymmetry. However,

the geometric controller successfully arrested this transient drift and recovered the trajectory. This rapid stabilization was directly enabled by the PINN continuously converging on the offset parameters (Figs. 5 and 6) and dynamically feeding them back into the control allocation matrix. While this represents a partial parameter convergence, it proved entirely sufficient to restore system stability. By dynamically identifying and compensating for the majority of the disturbance torque, the updated allocation matrix effectively restored the baseline controller's authority, allowing it to seamlessly reject the remaining unmodeled dynamics without saturating the rotors.

Furthermore, the network successfully converged upon the payload mass prediction. However, as observed in Fig. (2), the mass estimation exhibited cyclical oscillations after the 20-second mark. These periodic fluctuations correspond directly to the dynamic tracking maneuvers, indicating that the network's mass estimation remains coupled to the multi-agent system's changing inertial movements during maneuvers.

TABLE II
PERFORMANCE METRICS ACROSS MULTIPLE EPISODES

Quality	Episode 1	Episode 5	Episode 10	Units
RMSE Total Loss	0.5462	1.5972	2.2172	-
RMSE Data Loss	0.0023	0.0017	0.0176	-
RMSE Physics Loss	0.3155	1.4201	2.042	-
Offset X RMSE	0.0153	0.0478	0.0518	meters
Offset Y RMSE	0.0470	0.0762	0.0543	meters
Trajectory Loss RMSE	0.4129	0.3757	0.3643	meters
Trajectory x-axis RMSE	0.2443	0.1521	0.1198	meters
Trajectory y-axis RMSE	0.2799	0.2946	0.2954	meters
Trajectory z-axis RMSE	0.1802	0.1768	0.1763	meters
Avg PINN Compute Time	37.59	34.96	37.18	ms
Avg PINN Compute Time	26.6	28.6	26.9	Hz

TABLE III
2nd EPISODE RESULTS

Quality	Value	Unit
RMSE Total Loss	0.6862	-
RMSE Data Loss	0.0016	-
RMSE Physics Loss	0.5230	-
Offset X RMSE	0.1564	meters
Offset Y RMSE	0.0901	meters
Trajectory Loss RMSE	0.4129	meters
Trajectory x-axis RMSE	0.2443	meters
Trajectory y-axis RMSE	0.2799	meters
Trajectory z-axis RMSE	0.1802	meters
Avg PINN Compute Time	37.59	ms
Avg PINN Compute Time	26.6	Hz

VII. CONCLUSION AND FUTURE WORK

This paper presented a novel parameter estimation framework for rigid multi-quadrotor systems utilizing an in-the-loop Physics-Informed Neural Network. By formulating a loss function that incorporates Euler-Lagrange and Euler-Poincaré equations directly on the $SO(3)$ manifold, the network guarantees globally valid rigid-body constraints while eliminating minimal-coordinate gimbal lock. Furthermore, by optimizing over a receding temporal window using continuous gradient descent, the estimator bypasses the strict full-rank

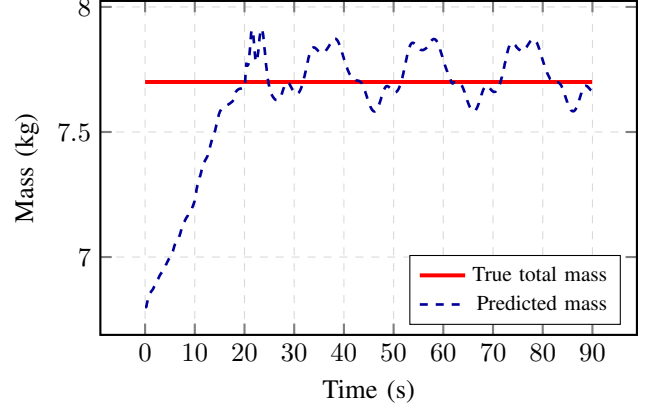


Fig. 2. Mass prediction over-time at 2st episode

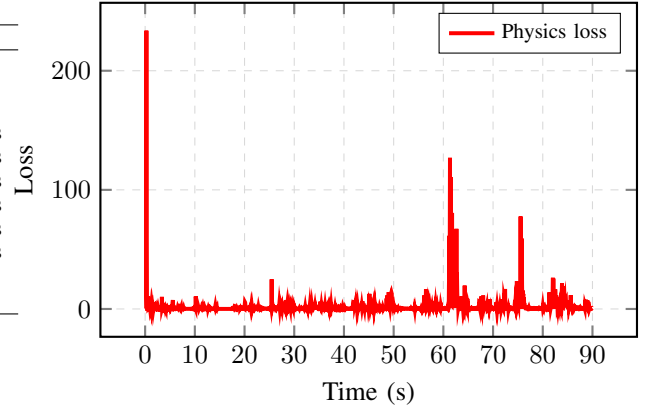


Fig. 3. Physics loss over-time at 2st episode

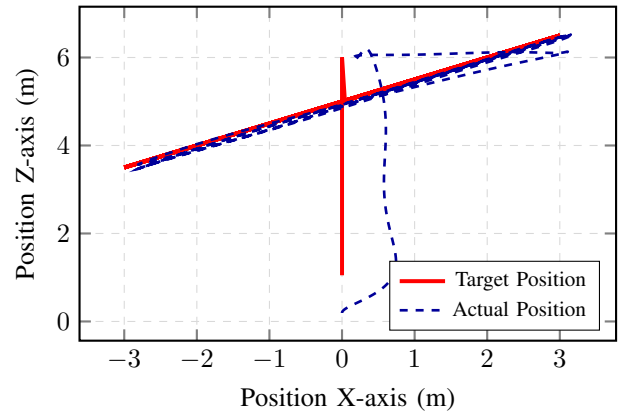


Fig. 4. Body frame position on XZ-axis at 2st episode

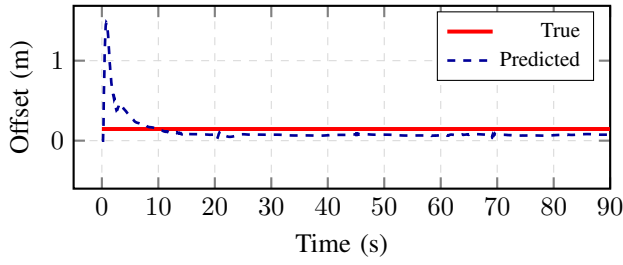


Fig. 5. Predicted payload offset over-time on X-axis at 2nd episode

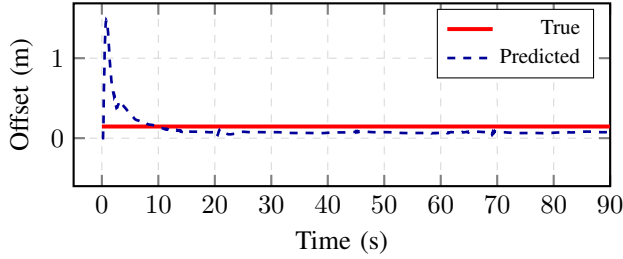


Fig. 6. Predicted payload offset over-time on Y-axis at 2nd episode

data requirements and matrix inversion vulnerabilities inherent to classical adaptive controllers.

Simulation results demonstrate that integrating this rank-free estimator directly into the control allocation matrix allows the system to rapidly identify spatial mass asymmetry, arrest transient lateral drift, and maintain stable trajectory tracking without requiring persistent excitation or multi-episode training. Even with partial parameter convergence, the dynamic thrust reallocation effectively restores the baseline controller's authority to reject unmodeled disturbance torques. Future research will extend this methodology to dynamic underactuated slung loads, incorporating coupled pendulum dynamics into the PINN's physical residuals to actively dampen payload oscillations.

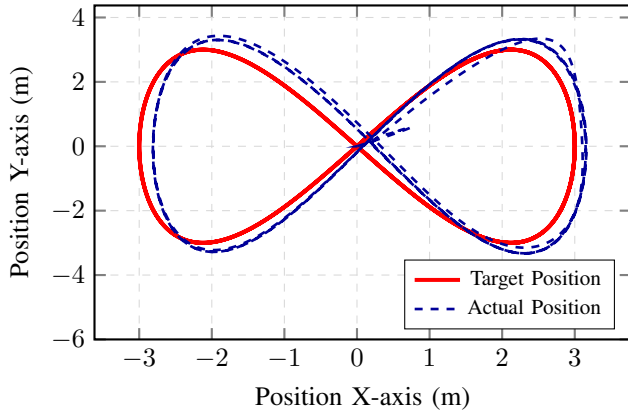


Fig. 7. Target position and actual position at 2th episode

REFERENCES

- [1] R. Guebsi, S. Mami, and K. Chokmani, "Drones in Precision Agriculture: A Comprehensive Review of Applications, Technologies, and Challenges," *Drones*, vol. 8, no. 11, p. 686, Nov. 2024. [Online]. Available: <https://www.mdpi.com/2504-446X/8/11/686>
- [2] Y. H. Tan, S. Lai, K. Wang, and B. M. Chen, "Cooperative Heavy Lifting Using Unmanned Multi-Agent Systems," in *2018 IEEE 14th International Conference on Control and Automation (ICCA)*. Anchorage, AK: IEEE, Jun. 2018, pp. 1119–1126. [Online]. Available: <https://ieeexplore.ieee.org/document/8444215/>
- [3] J. Zeng, A. M. Gimenez, E. Vinitzky, J. Alonso-Mora, and S. Sun, "Decentralized Aerial Manipulation of a Cable-Suspended Load using Multi-Agent Reinforcement Learning," Nov. 2025, arXiv:2508.01522 [cs]. [Online]. Available: <http://arxiv.org/abs/2508.01522>
- [4] B. Peng, B. Su, K. Yi, H. Wu, and D. He, "Modeling and Analysis of Four-UAV-Slung-Load System," in *2023 35th Chinese Control and Decision Conference (CCDC)*. Yichang, China: IEEE, May 2023, pp. 4289–4294. [Online]. Available: <https://ieeexplore.ieee.org/document/10327396/>
- [5] D. Mellinger, M. Shomin, N. Michael, and V. Kumar, "Cooperative grasping and transport using multiple quadrotors," in *Distributed Autonomous Robotic Systems*. Springer, 2013, pp. 545–558.
- [6] H. Ye, H. Yu, B. Liu, J. Han, Y. Fang, and X. Liang, "Nonlinear Robust Control for Dual-UAV Collaborative Aerial Transportation System," in *2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT)*. Jiaxing, China: IEEE, Nov. 2023, pp. 870–875. [Online]. Available: <https://ieeexplore.ieee.org/document/10528496/>
- [7] C.-A. Lee and T.-H. Cheng, "Concurrent Learning for Cooperative UAV Transportation of Unknown Payloads," *IEEE Open Journal of Control Systems*, vol. 4, pp. 187–198, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10797683/>
- [8] U. Farid, B. Khan, S. M. Ali, and Z. Ullah, "A Digital Twin Model for UAV Control to Lift Irregular-Shaped Payloads Using Robust Model Predictive Control," *Machines*, vol. 13, no. 11, p. 1069, Nov. 2025. [Online]. Available: <https://www.mdpi.com/2075-1702/13/11/1069>
- [9] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational physics*, vol. 378, pp. 686–707, 2019.
- [10] R. Mojtani, M. Balajewicz, and P. Hassanzadeh, "Kolmogorov n-width and Lagrangian physics-informed neural networks: A causality-conforming manifold for convection-dominated PDEs," *Computer Methods in Applied Mechanics and Engineering*, vol. 404, p. 115810, Feb. 2023. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0045782522007666>
- [11] S. Lakshminarayana, S. Sthapit, and C. Maple, "Application of Physics-Informed Machine Learning Techniques for Power Grid Parameter Estimation," *Sustainability*, vol. 14, no. 4, p. 2051, Feb. 2022. [Online]. Available: <https://www.mdpi.com/2071-1050/14/4/2051>
- [12] G. Serrano, M. Jacinto, J. Ribeiro-Gomes, J. Pinto, B. J. Guerreiro, A. Bernardino, and R. Cunha, "Physics-Informed Neural Network for Multirotor Slung Load Systems Modeling," May 2024, arXiv:2405.09428 [cs]. [Online]. Available: <http://arxiv.org/abs/2405.09428>
- [13] M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, Feb. 2019. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0021999118307125>
- [14] G. Haghighatdoost, "Euler Poincare Dynamics and Control on Lie Groupoids," May 2026, arXiv:2509.20044 [math.DS]. [Online]. Available: <http://arxiv.org/abs/2509.20044>